Centro de Comercio y Servicios

Análisis y Desarrollo de Software

Instructor: Raúl Gómez

Hilo de ejecución

Los SO modernos incorporan el concepto de hilo de ejecución (thread). En un sistema multihilo, un proceso va a poder realizar varias tareas a la vez, cada una en un hilo diferente. Los diferentes hilos de un proceso lo comparten todo: variables, código, permisos, archivos abiertos, etc.

Hilo de ejecución

Thread:



Hilo de ejecución

Cuando se trabaja con varios hilos, estos pueden acceder a las variables de forma simultánea. Hay que tener cuidado de que un hilo no modifique el valor de una variable mientras otro hilo la está leyendo. Este problema se resuelve en Java definiendo secciones críticas mediante la palabra reservada synchronized.

Hilos de ejecución en Android

Cuando se lanza una nueva aplicación, el sistema crea un nuevo hilo de ejecución (thread) para esta aplicación, conocido como hilo principal. Este hilo es muy importante, dado que se encarga de atender a los eventos de los distintos componentes. Es decir, este hilo ejecuta los métodos onCreate(), onDraw(), onKeyDown(), etc.

Hilos de ejecución en Android

El hilo principal también es el responsable de atender a los eventos generados desde la interfaz de usuario. Por esta razón, al hilo principal también se lo conoce como hilo de la interfaz de usuario.

El sistema no crea un hilo independiente cada vez que se crea un nuevo componente. Es decir, todas las actividades y servicios de una aplicación son ejecutados por el hilo principal de la aplicación.

Hilos de ejecución en Android

Cuando una aplicación ha de realizar trabajo intensivo como respuesta a una interacción de usuario, hay que tener cuidado porque es posible que la aplicación no responda de forma adecuada. Por ejemplo, la realización de algún cálculo que lleve bastante tiempo, si se hace en el hilo de ejecución principal, este quedará bloqueado a la espera de que termine el cálculo.

Hilos de ejecución en Android

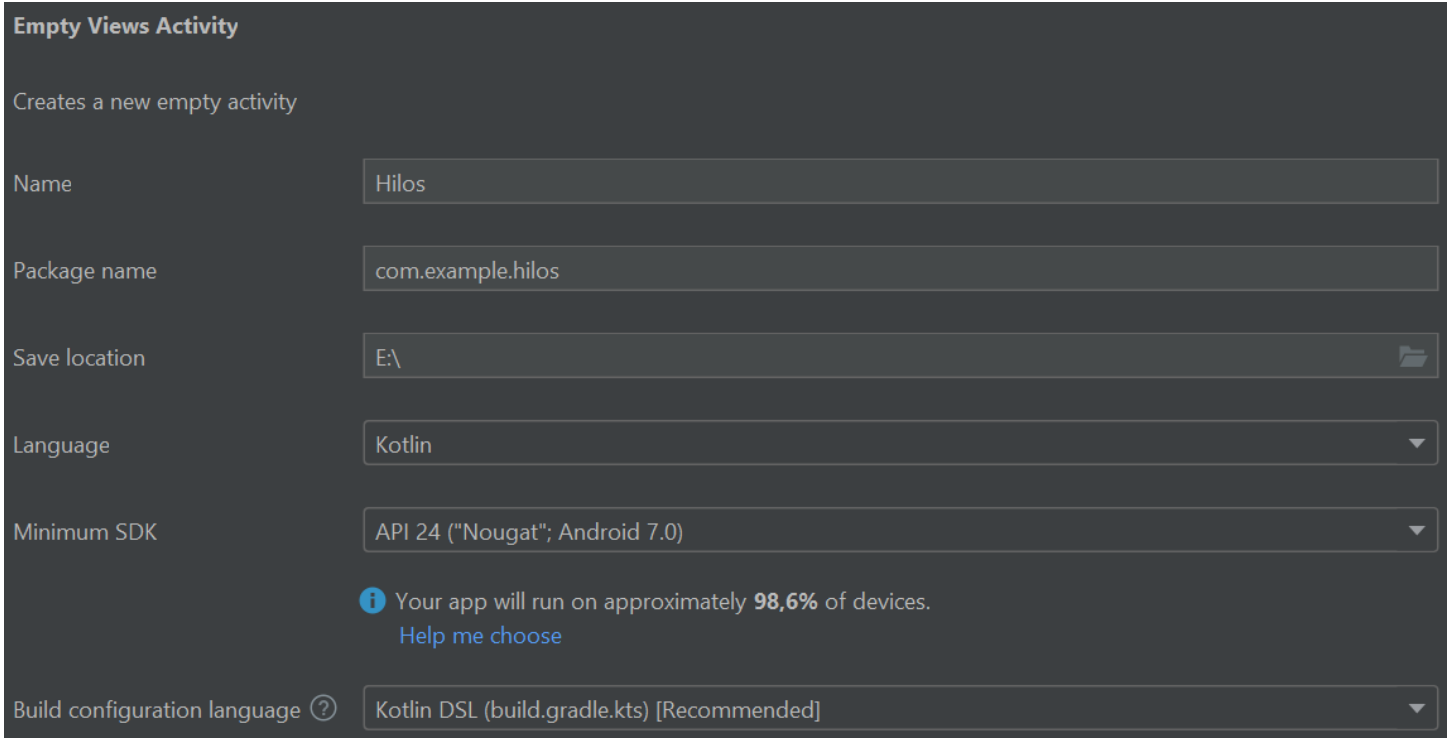
La solución en estos casos es crear un nuevo hilo de ejecución, para que realice este trabajo intensivo. De esta forma no bloqueamos el hilo principal, que puede seguir atendiendo a los eventos de usuario. Es decir, cuando se esté implementando un método que es ejecutado por el hilo principal (suelen empezar por on…).

Hilos de ejecución en Android

Las herramientas de la interfaz de usuario de Android han sido diseñadas para ser ejecutadas desde un único hilo de ejecución, el hilo principal. Por lo tanto, no se permite manipular la interfaz de usuario desde otros hilos de ejecución.

Hilo de ejecución

Crear proyecto



Hilo de ejecución

MainActivity.kt:

```
package com.example.hilos

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.os.SystemClock
import android.view.View
import android.widget.EditText
import android.widget.TextView

class MainActivity : AppCompatActivity() {
    private lateinit var entrada: EditText
    private lateinit var salida: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        entrada = findViewById(R.id.entrada)
        salida = findViewById(R.id.salida)
    }

    fun calcularOperacion(view: View) {
        val n = entrada.text.toString().toInt()
        salida.append("$n! = ")
        val res = factorial(n)
        salida.append("$res\n")
    }

    fun factorial(n: Int): Int {
        var res = 1
        for (i in 1..n) {
            res *= i
            SystemClock.sleep(1000)
        }
        return res
    }
}
```

Hilo de ejecución

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <EditText
            android:id="@+id/entrada"
            android:layout_width="100dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:ems="10"
            android:inputType="number|numberDecimal"
            android:text="5" >
            <requestFocus></requestFocus>
        </EditText>

        <Button
            android:id="@+id/button"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:onClick="calcularOperacion"
            android:text="Calcular factorial" />

    </LinearLayout>

    <TextView
        android:id="@+id/salida"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_weight="1"
        android:text=" "
        android:textAppearance="?android:attr/textAppearanceMedium"/>
</LinearLayout>
```

Hilo de ejecución

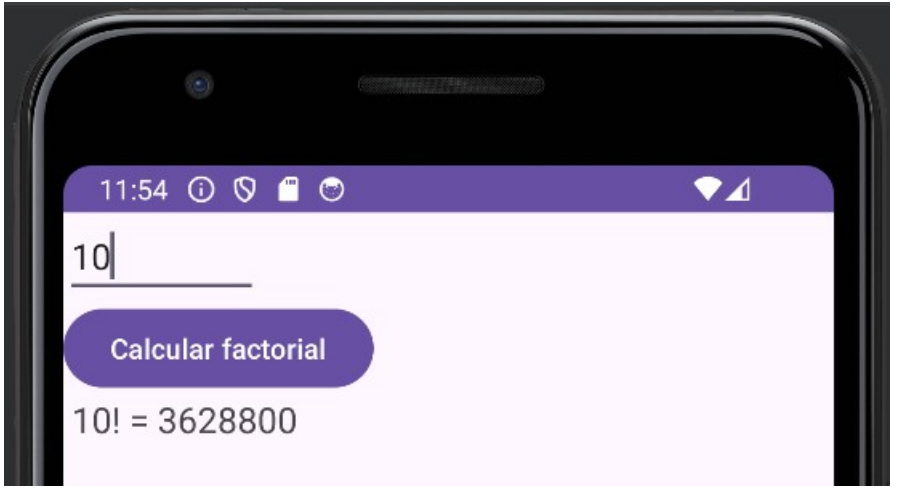
El método calcularOperacion() se llamará cuando se pulse el botón. Comienza obteniendo el valor entero introducido en entrada y muestra la operación a realizar por el TextView salida. Luego, se llama al factorial() y se muestra el resultado.

Hilo de ejecución

Esta operación se va a computar de forma muy rápida y apenas se bloqueará el hilo principal unas milésimas de segundo. Se va a simular que se realizan un gran número de operaciones en cada pasada del bucle. Para ello se llamará a SystemClock.sleep(1000), que bloqueará el hilo durante 1000 ms (1 segundo).

Hilo de ejecución

Resultado:



Hilo de ejecución

Mensajes del tipo “La aplicación no responde” son frecuentes en Android. Aparecen cuando el hilo principal se bloquea demasiado tiempo.

Hilo de ejecución

Siempre que se tenga que ejecutar un método que requiera bastante tiempo de ejecución, no se podrá ejecutarlo en el hilo principal. Dado que este hilo ha de estar siempre disponible para atender a los eventos generados por el usuario, nunca debe ser bloqueado.

Hilo de ejecución

MainActivity.kt:

```
inner class MiThread(private val n: Int) : Thread() {
    override fun run() {
        val res = factorial(n)
        runOnUiThread { salida.append("$res\n") }
    }
}
```

Hilo de ejecución

MainActivity.kt:

```
fun calcularOperacion(view: View) {
    val n = entrada.text.toString().toIntOrNull() ?: return
    salida.append("$n! = ")
    MiThread(n).start()
}
```

Hilo de ejecución

El método run() contiene el código que se quiere que el hilo ejecute.

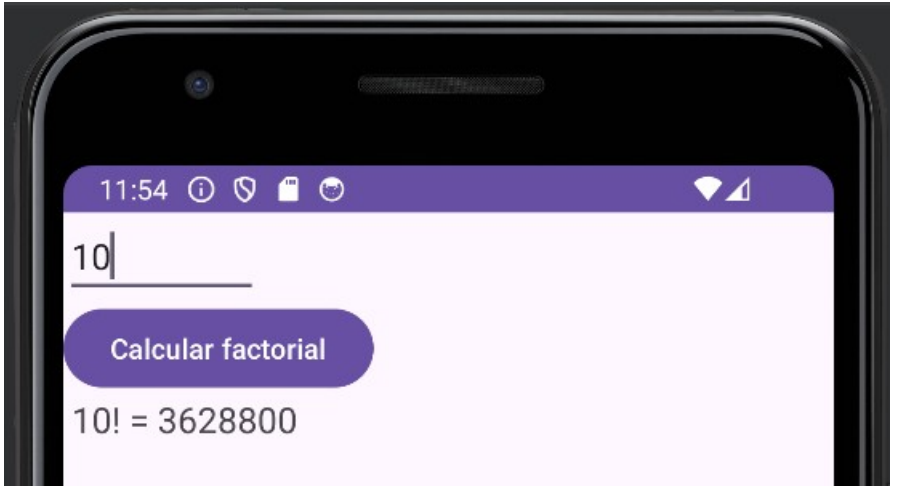
La llamada al método start() ocasionará que se ejecute el método run() del hilo. La llamada a start() es asíncrona, es decir, se continúa ejecutando las instrucciones siguientes, sin esperar a que el método run() concluya.

Hilo de ejecución

El método run() llama a factorial() y muestra el resultado por pantalla. Para realizar la operación se necesita el parámetro de entrada n. Este no puede incorporarse al método run(), dado que ha de ser sobrescrito (override) sin alteración alguna.

Hilo de ejecución

Resultado:



Secciones críticas

Cuando se realiza una aplicación que ejecuta varios hilos de ejecución, hay que prestar especial cuidado a que ambos hilos puedan acceder de forma simultánea a los datos. Cuando se limitan a leer las variables, no suele haber problemas. El problema aparece cuando un hilo está modificando algún dato y justo en ese instante se pasa a ejecutar un segundo hilo que ha de leer esos datos.

Secciones críticas

Este segundo hilo va a encontrar unos datos a medio modificar, lo que posiblemente cause errores en su interpretación.

La clase AsyncTask

Una tarea asincrónica (AsyncTask) permite realizar un cálculo o proceso que se ejecuta en un hilo secundario y cuyo resultado se quiere que se publique en el hilo de la interfaz de usuario.

Hilo de ejecución

MainActivity.kt:

```
fun calcularOperacion(view: View) {
    val n = entrada.text.toString().toIntOrNull() ?: return
    salida.append("$n! = ")

    lifecycleScope.launch {
        val resultado = factorial(n)
        salida.append("$resultado\n")
    }
}
```

Aplicación asteroides

MainActivity.kt (código completo):

```
package com.example.hilos

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.view.View
import android.widget.EditText
import android.widget.TextView
import androidx.lifecycle.lifecycleScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

class MainActivity : AppCompatActivity() {
    private lateinit var entrada: EditText
    private lateinit var salida: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        entrada = findViewById(R.id.entrada)
        salida = findViewById(R.id.salida)
    }

    fun calcularOperacion(view: View) {
        val n = entrada.text.toString().toIntOrNull() ?: return
        salida.append("$n! = ")

        lifecycleScope.launch {
            val resultado = factorial(n)
            salida.append("$resultado\n")
        }
    }

    private suspend fun factorial(n: Int): Int = withContext(Dispatchers.Default) {
        var res = 1
        for (i in 1..n) {
            res *= i
            delay(1000)
        }
        res
    }
}
```

Cuadro de progreso

Si se está realizando una tarea que puede prolongarse en el tiempo, resulta muy importante mostrar al usuario cuándo empieza su progreso y cuándo termina.

Cuadro de progreso

activity_main.xml:

```
<ProgressBar
    android:id="@+id/barraProgreso"
    style="?android:attr/progressBarStyleHorizontal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:visibility="gone"
    android:max="10"
    android:progress="0"/>
```

Aplicación asteroides

MainActivity.kt (código completo):

```
package com.example.hilos

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.view.View
import android.widget.EditText
import android.widget.ProgressBar
import android.widget.TextView
import androidx.lifecycle.lifecycleScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

class MainActivity : AppCompatActivity() {
    private lateinit var entrada: EditText
    private lateinit var salida: TextView
    private lateinit var barraProgreso: ProgressBar

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        entrada = findViewById(R.id.entrada)
        salida = findViewById(R.id.salida)
        barraProgreso = findViewById(R.id.barraProgreso)
    }

    fun calcularOperacion(view: View) {
        val n = entrada.text.toString().toIntOrNull() ?: return
        salida.append("$n! = ")

        barraProgreso.max = n
        barraProgreso.progress = 0
        barraProgreso.visibility = View.VISIBLE

        lifecycleScope.launch {
            val resultado = factorial(n)
            salida.append("$resultado\n")
            barraProgreso.visibility = View.GONE
        }
    }

    private suspend fun factorial(n: Int): Int = withContext(Dispatchers.Default) {
        var res = 1
        for (i in 1..n) {
            res *= i
            delay(1000)

            withContext(Dispatchers.Main) {
                barraProgreso.progress = i
            }
        }
        res
    }
}
```

Cuadro de progreso

Resultado:



Eventos de usuario

Android captura los distintos eventos generados por el usuario de forma homogénea y se los pasa a la clase encargada de tratarlos. Por lo general, va a ser un objeto tipo View el que recogerá estos eventos por medio de dos técnicas alternativas: los escuchadores de eventos (Event Listener) y los manejadores de eventos (Event Handler).

Escuchador de eventos

La clase View define varias interfaces conocidas como escuchadores de eventos o Event Listener. Estas interfaces definen un método callback al que el sistema llamará cuando se produzca una acción determinada. Cuando queramos que un objeto reaccione ante un evento de una vista, se tiene que implementar la interfaz correspondiente al evento y registrar el objeto como escuchador de ese evento.

Escuchador de eventos

View.OnClickListener: Se llama cuando el usuario selecciona la vista. Se puede utilizar cualquier medio, como la pantalla táctil, las teclas de navegación.

View.OnLongClickListener: Se llama cuando el usuario selecciona la vista y la mantiene seleccionada durante más de un segundo.

Escuchador de eventos

View.OnFocusChangeListener: Se llama cuando el usuario navega dentro o fuera de un elemento.

View.OnKeyListener: Se llama cuando se pulsa o se suelta una tecla del dispositivo. La vista que tenga el foco en ese momento es la que generará el evento.

View.OnTouchListener: Se llama cuando se pulsa, se suelta o se desplaza en la pantalla táctil.

Escuchador de eventos

View.OnCreateContextMenuListener: Se llama cuando se va a crear un menú de contexto asociado a una vista.

Manejadores de eventos

Si se crea un descendiente de la clase View, se podrá utilizar varios métodos callback directamente usados como manejadores de eventos (Event Handlers). Se puede sobrescribir uno de estos métodos cuando se quiera que el código reaccione ante un evento determinado.

La pantalla táctil

Es utilizada como método principal de entrada. Se puede seleccionar, arrastrar y soltar cualquier elemento de la pantalla de forma sencilla.

El manejo básico de la pantalla táctil pasa por definir el método onTouchEvent en una clase View (o implementar la interfaz OnTouchListener en otras clases). Este método nos pasará en un parámetro un objeto de la clase MotionEvent con información sobre el evento.

La pantalla táctil

getAction(): Tipo de acción realizada. Puede ser ACTION_DOWN, ACTION_MOVE, ACTION_UPo ACTION_CANCEL; y algunos más para gestionar pantallas multitouch.

getX(), getY(): Posición de la pulsación.

getDownTime(): Tiempo en ms en que el usuario presionó por primera vez en una cadena de eventos de posición.

getEventTime(): Tiempo en ms del evento actual.

getPressure(): Estima la presión de la pulsación. El valor 0 es el mínimo, el valor 1 representa una pulsación normal.

getSize(): Valor escalado en 0 y 1 que estima el grosor de la pulsación.

La pantalla táctil

getAction(): Tipo de acción realizada. Puede ser ACTION_DOWN, ACTION_MOVE, ACTION_UPo ACTION_CANCEL; y algunos más para gestionar pantallas multitouch.

getX(), getY(): Posición de la pulsación.

getDownTime(): Tiempo en ms en que el usuario presionó por primera vez en una cadena de eventos de posición.

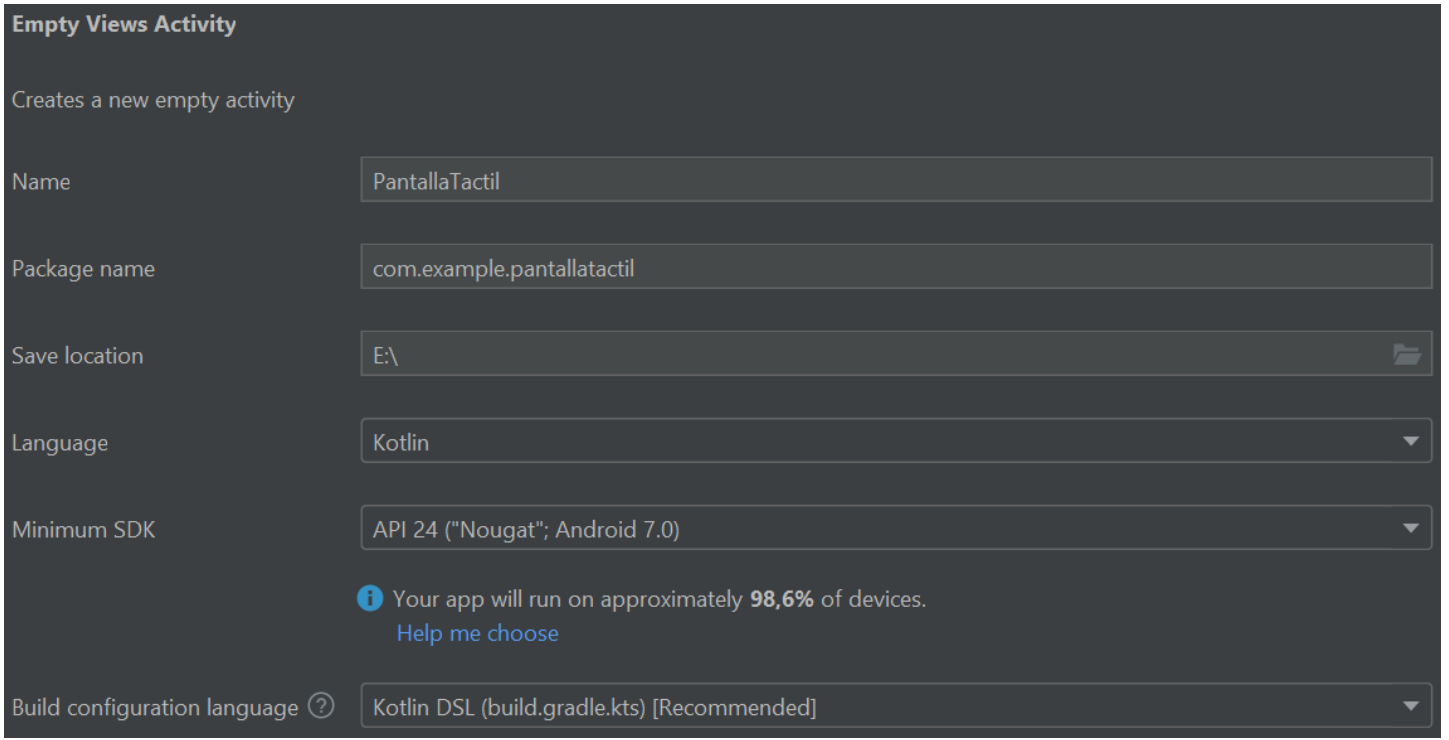
getEventTime(): Tiempo en ms del evento actual.

getPressure(): Estima la presión de la pulsación. El valor 0 es el mínimo, el valor 1 representa una pulsación normal.

getSize(): Valor escalado en 0 y 1 que estima el grosor de la pulsación.

La pantalla táctil

Crear proyecto



La pantalla táctil

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/TextViewEntrada"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_margin="2mm"
        android:layout_weight="1"
        android:background="#0000FF"
        android:gravity="center"
        android:text="Pulsa sobre esta vista"
        android:textSize="10pt" />

    <ScrollView
        android:id="@+id/scrollView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1">

        <TextView
            android:id="@+id/TextViewSalida"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:text="Pulsaciones:" />
    </ScrollView>

</LinearLayout>
```

La pantalla táctil

activity_main.xml:



La pantalla táctil

MainActivity.kt:

```
package com.example.pantallatactil

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.view.MotionEvent
import android.view.View
import android.widget.TextView

class MainActivity : AppCompatActivity(), View.OnTouchListener {
    private lateinit var salida: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        salida = findViewById(R.id.TextViewSalida)
        val entrada: TextView = findViewById(R.id.TextViewEntrada)
        entrada.setOnTouchListener(this)
    }

    override fun onTouch(v: View?, event: MotionEvent?): Boolean {
        salida.append("$event\n")
        return true
    }
}
```

La pantalla táctil

Resultado:



La pantalla táctil

action = MotionEvent.ACTION_DOWN = 0 significa que se ha pulsado sobre la pantalla,

action = MotionEvent.ACTION_UP = 1 significa que se ha soltado

action = MotionEvent.ACTION_MOVE = 2, que se está desplazando el dedo.

multi-touch

Las pantallas táctiles tienen la posibilidad de indicar la posición de varios punteros sobre la pantalla a un mismo tiempo.

Un objeto MotionEvent contiene información de todos estos punteros. Un puntero estará activo desde que se pulsa sobre la pantalla hasta que se deja de presionar. El número de punteros activos puede consultarse llamando al método getPointerCount(). Cada puntero tiene un id para identificarlo que se asigna cuando se produce la primera pulsación.

MotionEvent

ACTION_DOWN: Se pulsa en la pantalla sin que haya otro puntero activo.

ACTION_UP: Se deja de presionar el último puntero activo.

ACTION_MOVE: Cualquiera de los punteros activos se desplaza.

ACTION_CANCEL: Se cancela un gesture.

MotionEvent

ACTION_OUTSIDE: El puntero se sale de la vista.

ACTION_POINTER_DOWN: Se pulsa un nuevo puntero distinto al primero.

ACTION_POINTER_UP: Se deja de presionar un puntero pero no es el último.

multi-touch

MainActivity.kt:

```
override fun onTouch(v: View?, event: MotionEvent?): Boolean {
        event?.let {
            val acciones = arrayOf(
                "ACTION_DOWN", "ACTION_UP", "ACTION_MOVE",
                "ACTION_CANCEL", "ACTION_OUTSIDE", "ACTION_POINTER_DOWN",
                "ACTION_POINTER_UP"
            )

            val accion = it.action
            val codigoAccion = accion and MotionEvent.ACTION_MASK
            val iPuntero = (accion and MotionEvent.ACTION_POINTER_INDEX_MASK) shr MotionEvent.ACTION_POINTER_INDEX_SHIFT
            salida.append(acciones[iPuntero])

            for (i in 0 until it.pointerCount) {
                salida.append(
                    " puntero:${it.getPointerId(i)}" + " x:${it.getX(i)} y:${it.getY(i)}"
                )
            }
            salida.append("\n")
        }
        return true
    }
```

multi-touch

Para visualizar cada posible acción se ha creado un array con sus nombres. A continuación se averigua la acción en la variable accion. Esta nueva acción la ha podido hacer cualquier puntero de los activos o uno nuevo. En esta variable se codifican simultáneamente el código de la acción (los 8 bits menos significativos) y el índice de puntero que la ocasiona (los 8 bits siguientes).

multi-touch

Una vez obtenido el código de la acción, se muestra su nombre en la vista salida. Luego se hace un bucle para mostrar información de todos los punteros activos. El método getPointerCount() nos permite averiguar su número. Se va a recorrer los punteros activos con la variable i. Estos métodos dan información sobre el primer puntero activo que se pulsó, pero también indican un índice de puntero (getX(i), getSize(i), etc.) para obtener información sobre el resto de los punteros activos.

multi-touch

El método getPointerId(int indice) permite averiguar el identificador del puntero. No hay que confundir el índice de puntero con su identificador. El índice se asigna en función del orden en que fueron pulsados. El índice cero siempre es el más antiguo, el índice uno es el siguiente que se pulsó, etc. El índice de un puntero decrece a medida que los punteros anteriores a él dejan de estar activos. Por el contrario, el identificador de un puntero se asigna cuando se crea y permanece constante durante toda su vida. Será muy útil para seguir la pista de un determinado puntero. El método findPointerIndex(int id) permite averiguar el índice de un puntero a partir de su identificador.

multi-touch

MainActivity.kt (código completo):

```
package com.example.pantallatactil

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.view.MotionEvent
import android.view.View
import android.widget.TextView

class MainActivity : AppCompatActivity(), View.OnTouchListener {
    private lateinit var salida: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        salida = findViewById(R.id.TextViewSalida)
        val entrada: TextView = findViewById(R.id.TextViewEntrada)
        entrada.setOnTouchListener(this)
    }

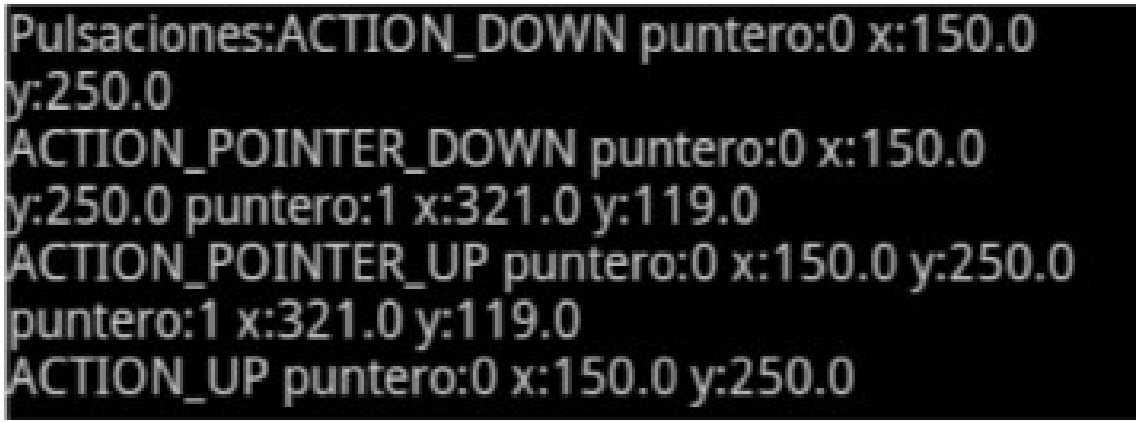
    override fun onTouch(v: View?, event: MotionEvent?): Boolean {
        event?.let {
            val acciones = arrayOf(
                "ACTION_DOWN", "ACTION_UP", "ACTION_MOVE",
                "ACTION_CANCEL", "ACTION_OUTSIDE", "ACTION_POINTER_DOWN",
                "ACTION_POINTER_UP"
            )

            val accion = it.action
            val codigoAccion = accion and MotionEvent.ACTION_MASK
            val iPuntero = (accion and MotionEvent.ACTION_POINTER_INDEX_MASK) shr MotionEvent.ACTION_POINTER_INDEX_SHIFT
            salida.append(acciones[iPuntero])

            for (i in 0 until it.pointerCount) {
                salida.append(
                    " puntero:${it.getPointerId(i)}" + " x:${it.getX(i)} y:${it.getY(i)}"
                )
            }
            salida.append("\n")
        }
        return true
    }
}
```

multi-touch

Resultado:



Sensores

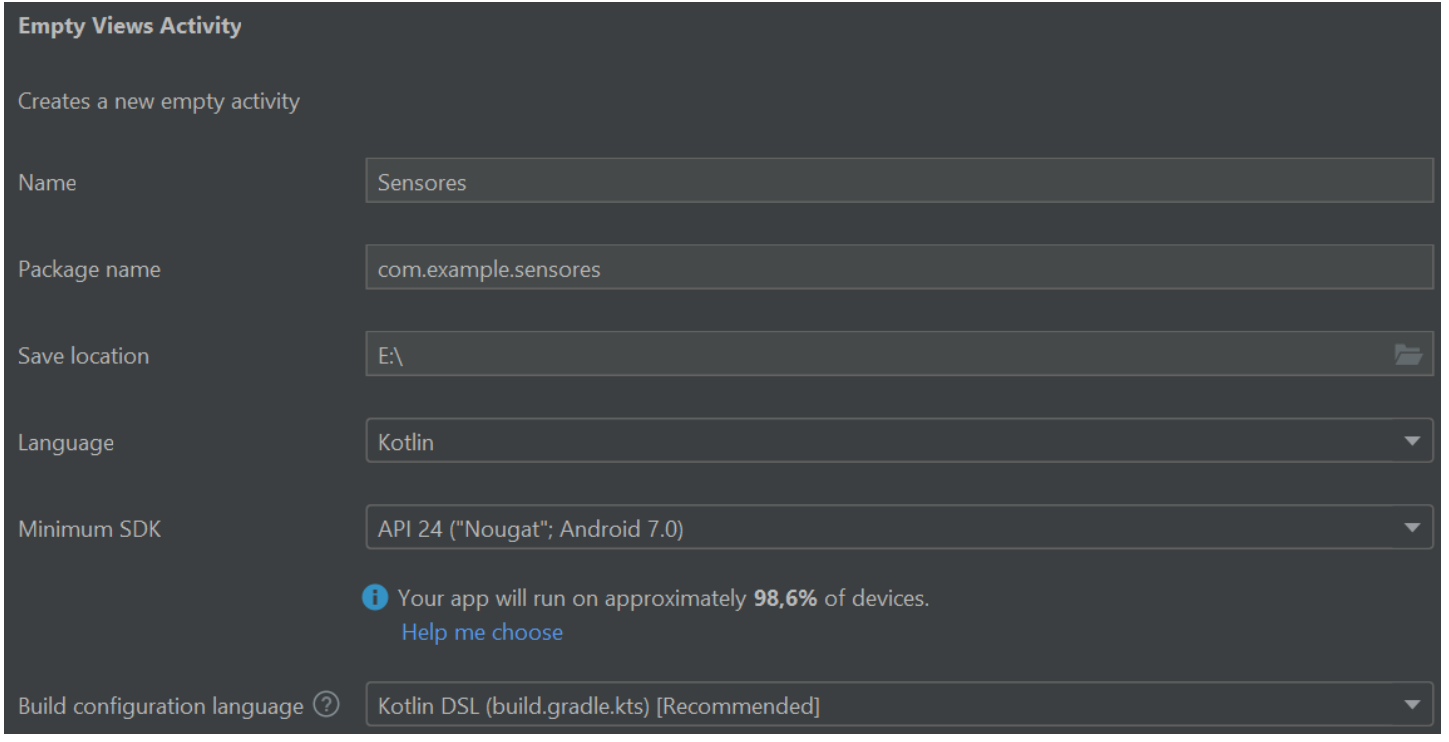
Bajo la denominación de sensores se engloba un conjunto de dispositivos con los que se puede obtener información del mundo exterior (en este conjunto no se incluye la cámara, el micrófono ni el GPS). Todos los sensores se manipulan de forma homogénea. Son los dispositivos de entrada más novedosos que incorpora Android y con ellos su puede implementar formas de interacción con el usuario.

Sensores

Android permite acceder a los sensores internos del dispositivo a través de las clases Sensor, SensorEvent, SensorManager y la interfaz SensorEventListener, del paquete android.hardware. La clase Sensor acepta varios tipos de sensores. Aunque los sensores disponibles varían en función del dispositivo utilizado.

Sensores

Crear proyecto



Sensores

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/salida"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Sensores

MainActivity.kt:

```
package com.example.sensores

import android.hardware.Sensor
import android.hardware.SensorManager
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.TextView

class MainActivity : AppCompatActivity() {
    private lateinit var salida: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        salida = findViewById(R.id.salida)

        val sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        val listaSensores: List<Sensor> = sensorManager.getSensorList(Sensor.TYPE_ALL)

        for (sensor in listaSensores) {
            log(sensor.name)
        }
    }

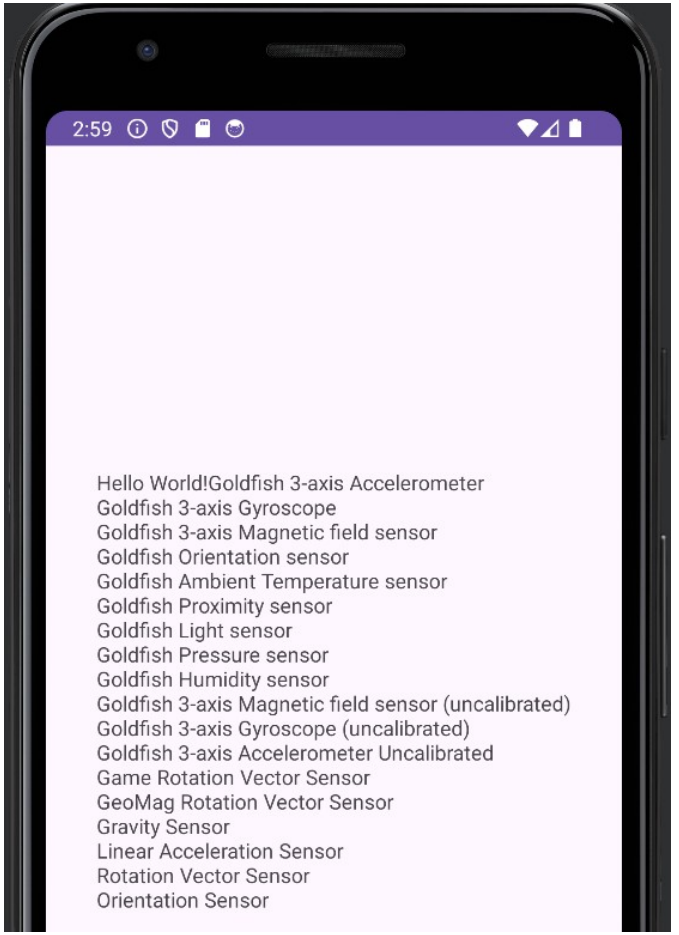
    private fun log(texto: String) {
        salida.append("$texto\n")
    }
}
```

Sensores

El método comienza indicando el layout de la actividad y obteniendo el TextView salida, donde se mostrarán los resultados. A continuación se va a utilizar el método getSystemService para solicitar al sistema servicios específicos. Este método pertenece a la clase Context (como es Activity, también es Context) y será muy utilizados para acceder a una gran cantidad de servicios del sistema. Al indicar como parámetro SENSOR_SERVICE, se indica que se quiere utilizar los sensores. Se hará a través del objeto sensorManager. En primer lugar se llama al método getSensorList() del objeto para que de listaSensores, una lista de objetos Sensor. La siguiente línea recorre todos los elementos de esta lista parar llamar a su método getName() para mostrar el nombre de sensor.

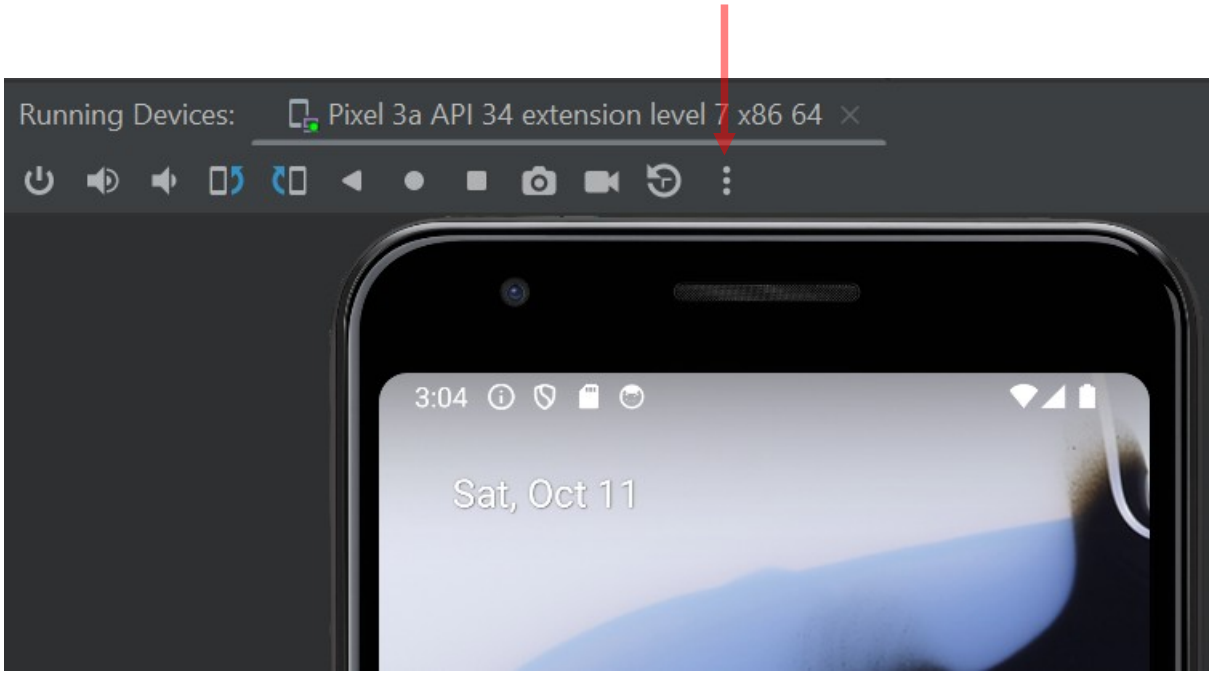
Sensores

Resultado:



Sensores

Mostrar la emulación de sensores:



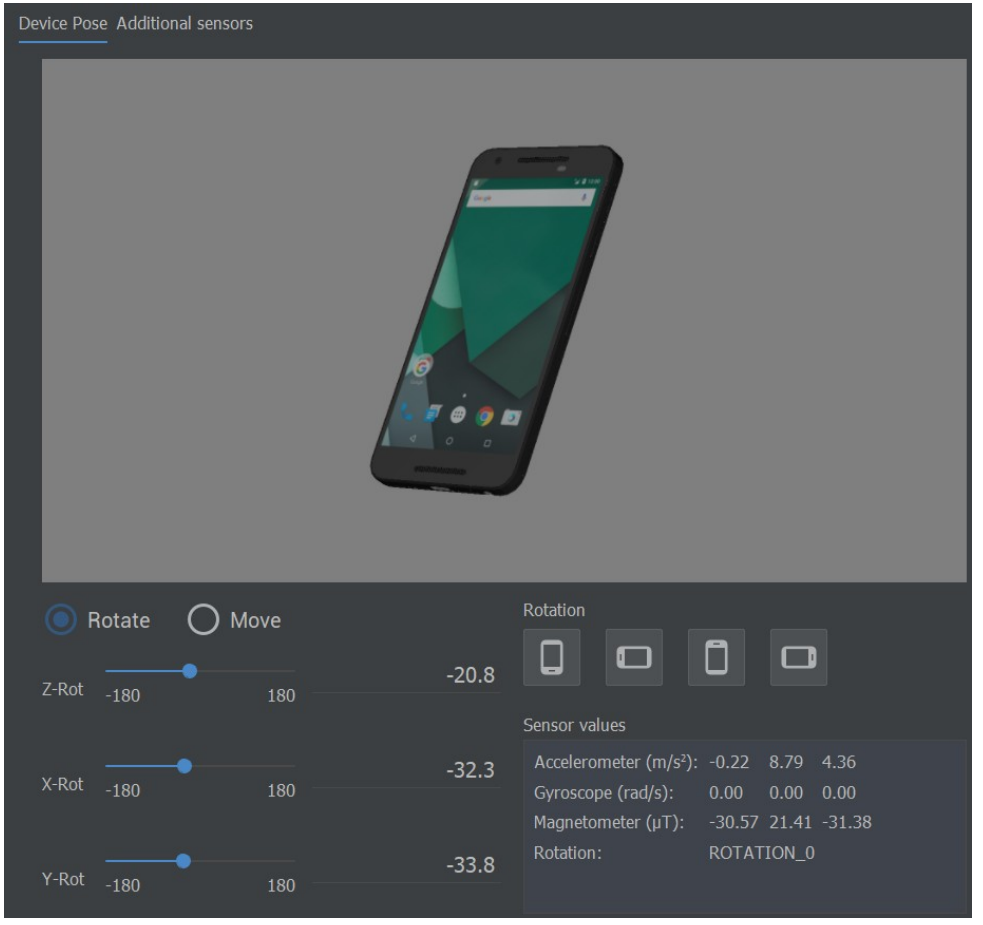
Sensores

Sensores virtuales



Sensores

Sensores virtuales



Sensores

MainActivity.kt:

```
listaSensores = sensorManager.getSensorList(Sensor.TYPE_ACCELEROMETER)
if (listaSensores.isNotEmpty()) {
    val accelerometerSensor = listaSensores[0]
    sensorManager.registerListener(this, accelerometerSensor, SensorManager.SENSOR_DELAY_UI)
}

listaSensores = sensorManager.getSensorList(Sensor.TYPE_MAGNETIC_FIELD)
if (listaSensores.isNotEmpty()) {
    val magneticSensor = listaSensores[0]
    sensorManager.registerListener(this, magneticSensor, SensorManager.SENSOR_DELAY_UI)
}

listaSensores = sensorManager.getSensorList(Sensor.TYPE_PROXIMITY)
if (listaSensores.isNotEmpty()) {
    val proximitySensor = listaSensores[0]
    sensorManager.registerListener(this, proximitySensor, SensorManager.SENSOR_DELAY_UI)
}
```

Sensores

Comenzamos pidiendo al sistema que nos dé todos los sensores de este tipo llamando a getSensorList(Sensor.TYPE_ACCELEROMETER). Si la lista no está vacía obtenemos el primer elemento (posición 0). Es necesario registrar cada tipo de sensor por separado para poder obtener información de él. El método registerListener() toma como primer parámetro un objeto que implemente la interfaz SensorEventListener.

Sensores

El segundo parámetro es el sensor que estamos registrando. Y el tercero indica al sistema con qué frecuencia nos gustaría recibir actualizaciones del sensor. Acepta cuatro posibles valores. De menor a mayor frecuencia tenemos: SENSOR_DELAY_NORMAL, SENSOR_DELAY_UI, SENSOR_DELAY_GAME y SENSOR_DELAY_FASTEST. Esta indicación sirve para que el sistema estime cuánta atención necesitan los sensores, pero no garantiza una frecuencia concreta.

Sensores

MainActivity.kt:

```
override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}

    override fun onSensorChanged(evento: SensorEvent?) {
        evento?.let {
            when (it.sensor.type) {
                Sensor.TYPE_ACCELEROMETER -> for (i in 0..2) log("Acelerómetro $i: ${it.values[i]}")
                Sensor.TYPE_MAGNETIC_FIELD -> for (i in 0..2) log("Magnetismo $i: ${it.values[i]}")
                else -> for (i in it.values.indices) log("Proximidad $i: ${it.values[i]}")
            }
        }
    }
```

Sensores

Cuando se implementa una interfaz, se está obligado a implementar todos sus métodos. En este caso son dos. Para onAccuracyChanged no se quiere ninguna acción específica, pero se tiene que incluir. Cuando un sensor cambie se llamará al método onSensorChanged. Aquí se comprueba qué sensor ha causado la llamada y se muestra los datos. Los posibles valores devueltos dependen del tipo de sensor.

Sensores

MainActivity.kt (código completo):

```
package com.example.sensores

import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.TextView

class MainActivity : AppCompatActivity(), SensorEventListener {
    private lateinit var salida: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        salida = findViewById(R.id.salida)

        val sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        var listaSensores: List<Sensor> = sensorManager.getSensorList(Sensor.TYPE_ALL)

        for (sensor in listaSensores) {
            log(sensor.name)
        }

        listaSensores = sensorManager.getSensorList(Sensor.TYPE_ACCELEROMETER)
        if (listaSensores.isNotEmpty()) {
            val accelerometerSensor = listaSensores[0]
            sensorManager.registerListener(this, accelerometerSensor, SensorManager.SENSOR_DELAY_UI)
        }

        listaSensores = sensorManager.getSensorList(Sensor.TYPE_MAGNETIC_FIELD)
        if (listaSensores.isNotEmpty()) {
            val magneticSensor = listaSensores[0]
            sensorManager.registerListener(this, magneticSensor, SensorManager.SENSOR_DELAY_UI)
        }

        listaSensores = sensorManager.getSensorList(Sensor.TYPE_PROXIMITY)
        if (listaSensores.isNotEmpty()) {
            val proximitySensor = listaSensores[0]
            sensorManager.registerListener(this, proximitySensor, SensorManager.SENSOR_DELAY_UI)
        }
    }

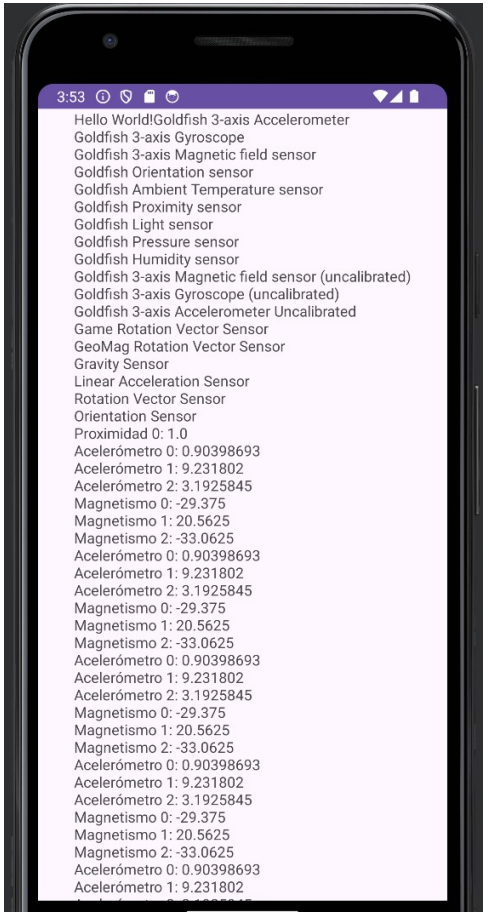
    private fun log(texto: String) {
        salida.append("$texto\n")
    }

    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}

    override fun onSensorChanged(evento: SensorEvent?) {
        evento?.let {
            when (it.sensor.type) {
                Sensor.TYPE_ACCELEROMETER -> for (i in 0..2) log("Acelerómetro $i: ${it.values[i]}")
                Sensor.TYPE_MAGNETIC_FIELD -> for (i in 0..2) log("Magnetismo $i: ${it.values[i]}")
                else -> for (i in it.values.indices) log("Proximidad $i: ${it.values[i]}")
            }
        }
    }
}
```

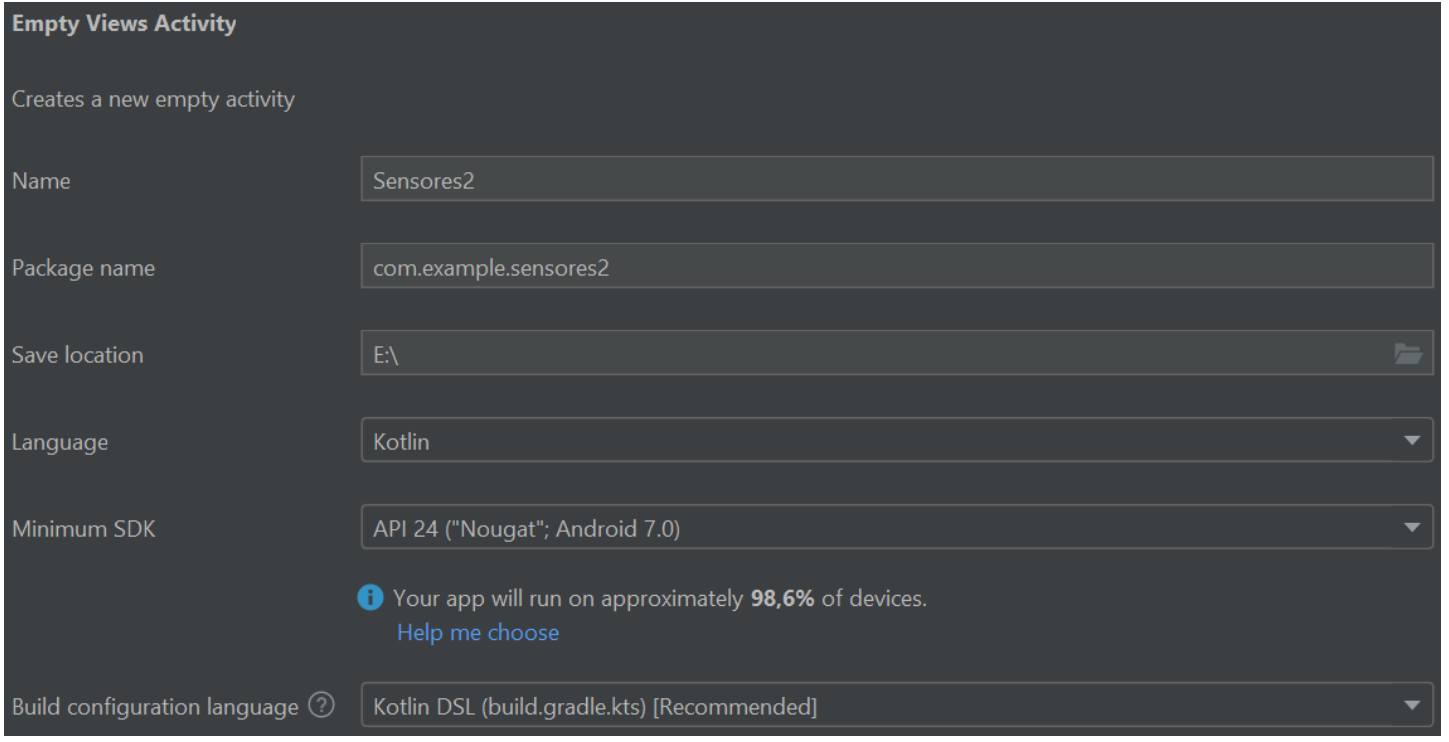
Sensores

Resultado:



Sensores

Crear proyecto



Sensores

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/raiz"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity" />
```

Sensores

MainActivity.kt:

```
package com.example.sensores2

import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.LinearLayout
import android.widget.TextView

class MainActivity : AppCompatActivity(), SensorEventListener {
    private lateinit var listaSensores: List<Sensor>
    private val aTextView = Array(50) { arrayOfNulls<TextView>(3) }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val raiz: LinearLayout = findViewById(R.id.raiz)
        val sm = getSystemService(SENSOR_SERVICE) as SensorManager
        listaSensores = sm.getSensorList(Sensor.TYPE_ALL)

        var n = 0
        for (sensor in listaSensores) {
            val mTextView = TextView(this)
            mTextView.text = sensor.name
            raiz.addView(mTextView)

            val nLinearLayout = LinearLayout(this)
            raiz.addView(nLinearLayout)

            // X, Y, Z labels + valores
            val etiquetas = listOf("X", "Y", "Z")
            for (i in 0..2) {
                val label = TextView(this)
                label.text = " ${etiquetas[i]}: "
                nLinearLayout.addView(label)

                val valor = TextView(this)
                valor.text = "?"
                valor.width = 87
                nLinearLayout.addView(valor)

                aTextView[n][i] = valor
            }

            sm.registerListener(this, sensor, SensorManager.SENSOR_DELAY_UI)
            n++
        }
    }

    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}

    override fun onSensorChanged(event: SensorEvent?) {
        synchronized(this) {
            event?.let { e ->
                for ((index, sensor) in listaSensores.withIndex()) {
                    if (e.sensor == sensor) {
                        for (i in 0 until minOf(e.values.size, 3)) {
                            aTextView[index][i]?.text = e.values[i].toString()
                        }
                        break
                    }
                }
            }
        }
    }
}
```

Sensores

Esta actividad utiliza el layout creado desde XML, que básicamente es un LinearLayot vacío. Se accede a este layout con el objeto raiz. Al layout se le irán añadiendo una serie de vistas según los sensores encontrados en el dispositivo. Por cada sensor se añade: un TextView con el nombre del sensor; un LinearLayout de tipo horizontal para contener, a su vez, un TextView con “X:”; un TextView con el valor del sensor en el eje X; un TextView con “Y”; un TextView con el valor del sensor en el eje Y; un TextView con “Z:”, y un TextView con el valor del sensor en el eje Z. Las referencias a los TextView donde se visualizarán los valores de los sensores se almacenan en el array aTextView[][], donde el primer índice identifica el número de sensor y el segundo, la dimensión X, Y o Z.

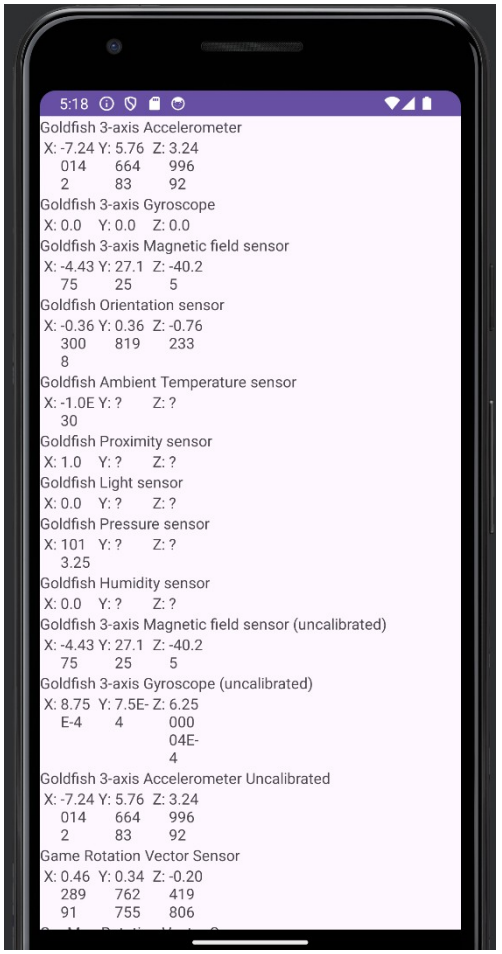
Sensores

En el método onSensorChanged() se hace un bucle para localizar el índice del sensor que ha cambiado y se modifican los TextViewcorrespondientes al sensor con los valores leídos.

No todos los sensores tienen tres dimensiones. Por ejemplo, en el caso del sensor de proximidad solo se cambiará en el valor de X. Mientras que otros, como los vectores de rotación, pueden devolver hasta 5 valores.

Sensores

Resultado:



Restricciones en segundo plano

Android presenta características para mejorar la administración de energía de los dispositivos. Estos cambios, garantizan que se proporcionen los recursos de sistema a las aplicaciones que más los necesiten. Una de las características es que Android limita la capacidad de las aplicaciones en segundo plano para acceder a entradas del usuario y datos de sensores.

Restricciones en segundo plano

Si una aplicación se ejecuta en segundo plano en un dispositivo, el sistema aplica a esta las siguientes restricciones:

La aplicación no puede acceder al micrófono ni a la cámara.

Los sensores que usan el modo de generación de informes continua, como acelerómetros y giroscopios, no reciben eventos.

Los sensores que usan los modos de generación de informes ante un cambio o de acción única no reciben eventos.

Si una aplicación debe detectar eventos de sensores, se deberá utilizar un servicio en primer plano.

Ejercicios prácticos

Ejercicio 1

Crear una aplicación con un Button y un TextView.

Cuando el usuario pulse el botón:

Se debe iniciar un hilo (Thread) que escriba en el TextView los números del 1 al 10, con un retardo de 1 segundo entre cada número.

Usar Thread.sleep(1000) y runOnUiThread { ... } para actualizar la interfaz desde el hilo secundario.

Ejercicios prácticos

Ejercicio 2

Diseñar una interfaz con un ProgressBar y un TextView.

Crear un hilo que simule una carga progresiva del 0 al 100, actualizando la barra y el texto cada 100 ms.

Ejercicios prácticos

Ejercicio 3

Crea tres hilos distintos:

Uno que imprima números pares.

Otro que imprima números impares.

Otro que imprima letras del alfabeto.

Hacer que todos se ejecuten al mismo tiempo y observar el resultado intercalado.

Ejercicios prácticos

Ejercicio 4

Implementar una clase MiTarea que realice una operación matemática (por ejemplo, calcular la suma de los primeros N números).

Ejercicios prácticos

Ejercicio 5

Crear un botón que al pulsarse lance una corrutina que imprima mensajes en la consola cada segundo durante 5 segundos.

Ejercicios prácticos

Ejercicio 6

Simular un sensor que genera datos aleatorios cada 500 ms.

Usar una corrutina que actualice la interfaz periódicamente con los nuevos valores hasta que el usuario presione un botón para detenerla.

Ejercicios prácticos

Ejercicio 7

Crear una aplicación con un TextView que muestre mensajes dependiendo del evento táctil:

ACTION_DOWN: “Pulsación iniciada”

ACTION_MOVE: “Desplazando el dedo”

ACTION_UP: “Pulsación finalizada”

Ejercicios prácticos

Ejercicio 8

Mostrar en un TextView las coordenadas x e y del punto donde el usuario toca la pantalla.

Ejercicios prácticos

Ejercicio 9

Diseñar una vista (View) personalizada que cambie su color de fondo cada vez que el usuario la toca.

Ejercicios prácticos

Ejercicio 10

Crear una ImageView (por ejemplo, una pelota).

Permitir al usuario arrastrarla libremente por la pantalla con el dedo.

Ejercicios prácticos

Ejercicio 11

Crear una vista personalizada que permita al usuario “dibujar” trazos en la pantalla al mover el dedo.

Cada trazo debe almacenarse y redibujarse cuando la vista se repinte.

Ejercicios prácticos

Ejercicio 12

Usar GestureDetector para reconocer:

Toque simple (tap)

Toque prolongado (long press)

Deslizamiento rápido (fling)

Ejercicios prácticos

Ejercicio 13

Crear una aplicación que detecte cuántos dedos hay en pantalla.

Mostrar sus posiciones x e y en tiempo real.

Ejercicios prácticos

Ejercicio 14

Dibujar trazos simultáneamente con varios dedos.

Cada dedo debe tener su propio color.

Ejercicios prácticos

Ejercicio 15

Crear una ImageView y permite al usuario acercar o alejar la imagen con el gesto de “pellizco”.

Ejercicios prácticos

Ejercicio 16

Crear una vista que gire según la posición de dos dedos.

Calcular el ángulo de rotación en función de las posiciones de los punteros.

Ejercicios prácticos

Ejercicio 17

Diseñar un lienzo donde cada dedo deje un punto de color distinto y se mueva al arrastrarlo.

Al soltar el dedo, el punto desaparece.

Ejercicios prácticos

Ejercicio 18

Controlar una imagen con:

Arrastre (para moverla)

Pinch (para escalar)

Rotación (para girar)

Ejercicios prácticos

Ejercicio 19

Crear una aplicación que cambie el color del fondo de la pantalla dependiendo de la distancia detectada por el sensor de proximidad:

Si el objeto está cerca → fondo rojo.

Si está lejos → fondo verde.

Ejercicios prácticos

Ejercicio 20

Diseñar una app que muestre la intensidad de luz ambiental en lux y cambie el brillo o el color del fondo dependiendo del valor.

Sensor: Sensor.TYPE_LIGHT

Ejercicios prácticos

Ejercicio 21

Usar acelerómetro + magnetómetro para calcular la orientación del dispositivo (azimut, pitch, roll) y mostrarla en pantalla.

Usar SensorManager.getRotationMatrix() y SensorManager.getOrientation().

Ejercicios prácticos

Ejercicio 22

Crear una brújula que apunte al norte magnético, representada con una ImageView que rota en función del azimut calculado.

imageView.rotation = -azimut.toFloat().

Ejercicios prácticos

Ejercicio 23

Crea una app donde el usuario pueda mover un círculo (ImageView) inclinando el dispositivo, usando los valores del acelerómetro.

Ejercicios prácticos

Ejercicio 24

Usar el Sensor.TYPE_STEP_DETECTOR o Sensor.TYPE_STEP_COUNTER para contar los pasos dados por el usuario y mostrar el total en un TextView.

Ejercicios prácticos

Ejercicio 25

Hacer que la pantalla cambie de color dependiendo tanto de la distancia (proximidad) como de la intensidad de luz:

Si está oscuro y cerca → negro.

Si está claro y lejos → blanco.

Ejercicios prácticos

Ejercicio 26

Detectar cuándo el usuario sacude el teléfono usando los valores del acelerómetro.

Calcular la magnitud del vector de aceleración y si supera cierto umbral, muestra un mensaje “¡Agitado!”.

Ejercicios prácticos

Ejercicio 27

Usar los valores de acelerómetro para detectar si el dispositivo está:

Boca arriba

Boca abajo

En posición vertical

Ejercicios prácticos

Ejercicio 28

Usar los sensores de paso, acelerómetro y giroscopio para crear una app que registre:

Número de pasos

Nivel de movimiento

Orientación promedio

Guarda los datos en una lista y muéstralos en un RecyclerView.